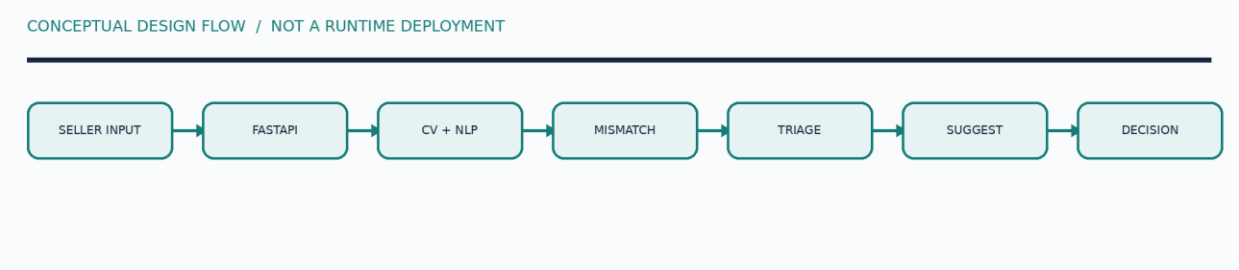
SMARTLISTING COPILOT

SYSTEM DESIGN ASSURANCE & TECHNICAL VALIDATION REPORT

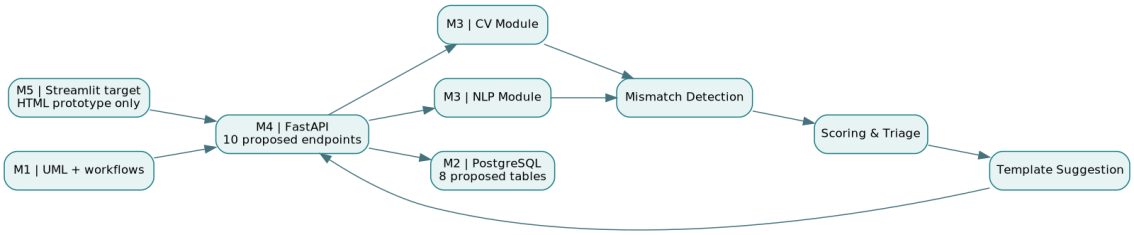
Cross-Artifact Verification | Integration Integrity | Requirements Traceability | Implementation Readiness

Prepared by Member 6 — Technical Design Validation & Quality Assurance

Version 1.0 • 30 October 2026 • Independent design review • Approval pending



REVIEW CLASSIFICATION: CONDITIONALLY READY — design contracts require resolution before integrated implementation acceptance. This report is not evidence of a deployed, trained, or tested application.


M3 | CV Module
M5 | Streamlit target
HTML prototype only
M3 | NLP Module
Mismatch Detection
M4 | FastAPI
10 proposed endpoints
Scoring & Triage
M1 | UML + workflows
M2 | PostgreSQL
8 proposed tables
Template Suggestion

1. Executive Engineering Assessment

The design is coherent at its main boundary: single-image fashion intake, product type and color analysis, parallel CV/NLP evidence, mismatch and triage, bounded template suggestion, and explicit seller Accept / Edit / Ignore without external publishing. The Member 3 architecture and Member 4 ten-endpoint proposal cover those proposed responsibilities.

Independent review captured 14 distinct design findings (9 Major, 2 Minor, 3 Observations). These are document-level assessment outcomes, not runtime defects. 41 executable-in-future test scenarios are PLANNED; zero were represented as executed by Member 6.

Decision: CONDITIONALLY READY for implementation planning and contract-first scaffolding; NOT accepted as integrated implementation. Mandatory gates: severity representation, category freeze, input/queue/retry decisions, real PostgreSQL checks and running API contract tests.

2. Document Control and Review Scope

Attribute	Value
Scope	Member 1–5 design artifacts and ZIP technical appendices
Original FRS and rubric	NOT PROVIDED as independent original files
Execution boundary	Static SQL review and contract comparison only; no PostgreSQL or FastAPI runtime
Submission date discrepancy	Team target 30 Oct; UML M1 milestone 6 Nov; approval needed
Decision authority	No supervisor or stakeholder approval inferred

3. Source Artifact Register

ID	Source	Version	Status / limitation
S01	Analysis & UML PDF	v1.0	Draft / approval pending
S02	Analysis & UML DOCX	v1.0	Draft / approval pending
S03	Database design PDF	v1.1	Proposal / pending
S04	SQL and diagram archive	undated archive	Static / SQLite surrogate evidence
S05	Architecture PDF	27 Oct version	Proposal / pending
S06	API specification PDF	v1.1	Proposal / pending
S07	UX/UI design PDF	v1.2	Proposal / pending
S08	HTML prototype	standalone	Demonstration only
S09	Files.zip alternative package	archive	Duplicate candidate
S10	FR traceability CSV inside SQL ZIP	secondary transcription	Secondary source
S11	Official marking rubric	not provided	NOT PROVIDED
S12	Original revised-scope proposal and signed baseline	not provided separately	NOT PROVIDED

The member 1 PDF is 84 pages while the supplied DOCX has an 80-page alternative rendering. Both share version/date claims, but neither is silently substituted for the other. The archive contains analysis alternatives; original FRS and official rubric are absent as standalone uploaded files.

4. Review Methodology

Review procedure: inventory filenames and archives; extract evidence from the provided designs; inspect SQL CREATE/CHECK/FOREIGN KEY/index statements; compare API method/path/payload and state vocabulary to persistence contracts; inspect actual HTML labels and simulated behavior; map FR statements from the explicitly secondary Member 2 CSV; classify gaps; assign owner, closure evidence, and planned regression scenarios. No independent model evaluation, user walkthrough, runtime PostgreSQL test, or server test was run.

5. Canonical System Design Baseline

In scope: Fashion Product Images (Small) dataset, exactly five categories to be selected after EDA, product type and color only. Proposed Streamlit Frontend → FastAPI Backend → CV and NLP → Mismatch Detection → Scoring & Triage → template Suggestion; relational Database plus nonrelational Model & Lexicon Store. Eight tables: supported_category, product_listing, analysis_run, visual_prediction, text_claim, analysis_finding, suggested_correction, seller_decision. The M4 API catalog specifies ten endpoints E-01 to E-10 under /api/v1.

6. Cross-Member Integration Architecture

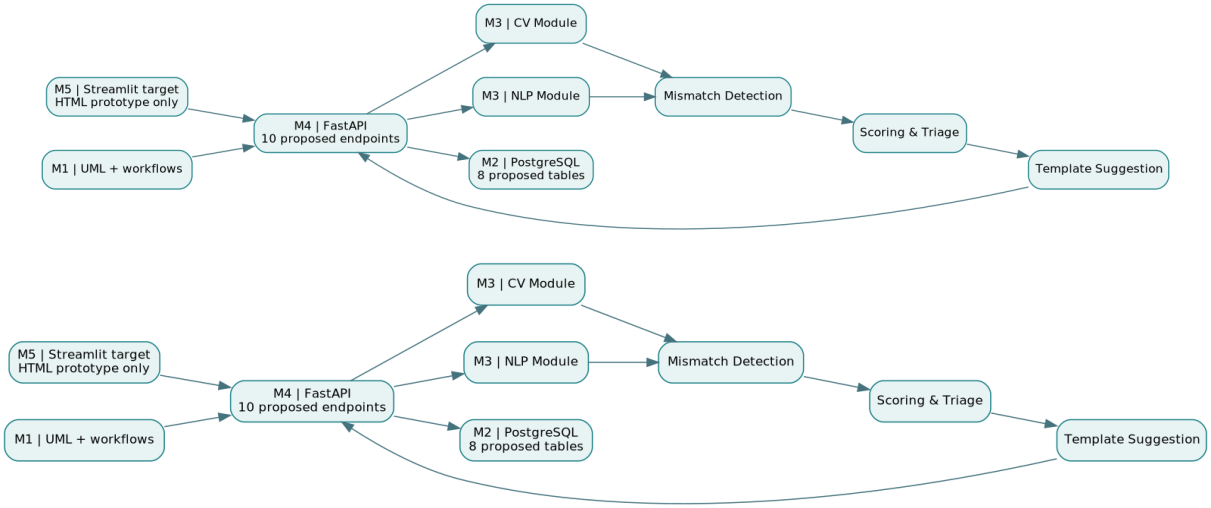


Figure A. Cross-member design and integration boundaries

Figure 1. Contract map synthesized from M3 PDF pp.1–4, M4 API catalog and M2 schema. Arrows show proposed integration, not running connections.

7. UML and Behavioral Design Audit

M1 has an 84-page UML and process design package, including behavioral diagrams, use cases, DFD, ERD, CRUD and schedule views (M1 PDF contents pp.2–3). The review found the proposed workflow direction compatible with M3 at a high level. State-transition integration remains conditional on matching failed/partial retry and latest-usable-run semantics; see F-09 and F-10. Milestone M1 appears as 6 November in the UML cover, while this team handoff is 30 October; neither date is rewritten.

8. Database Design Validation

The inspected ZIP contains 01_create_schema.sql, 02_constraints_indexes.sql, 03_seed_reference_data.sql, and 04_validation_queries.sql, plus ERD, dictionary and a Validation_Report. The schema defines eight tables with PK/FK/CHECK/UNIQUE constraints, review state and confidence bounds. SQL restricts analysis_finding.severity to lowercase critical/major/minor and nullable non-mismatch flags. The archive reports 140 static/SQLite-translation tests but explicitly not PostgreSQL runtime execution. These 140 are source claims, not Member 6 independently executed checks.

Table	FK count (static)	Review
supported_category	0	STATIC REVIEWED
product_listing	0	STATIC REVIEWED
analysis_run	1	STATIC REVIEWED
visual_prediction	2	STATIC REVIEWED
text_claim	1	STATIC REVIEWED
analysis_finding	1	STATIC REVIEWED
suggested_correction	1	STATIC REVIEWED
seller_decision	1	STATIC REVIEWED

Boundary checks in the test plan cover invalid confidence, unique decision, referential integrity, nullable predictions, partial-run and latest-usable-run selection, deletion behavior, and transaction rollback. These remain planned.

9. API Contract Verification

All ten E-01..E-10 entries are design-only. E-03 uses multipart/form-data and creates a listing/run; E-05 creates another run; E-08 records a single seller choice; E-09 returns latest completed/partial result by listing; E-10 returns image bytes. The synchronous request does not offer actual stage progress. Member 4 documents structured errors and decision conflicts; runtime conformance remains untested.

ID	Method	Path	Audit note
E-01	GET	/api/v1/health	Static contract only
E-02	GET	/api/v1/categories	Empty until EDA-approved categories
E-03	POST	/api/v1/listings/analyze	Multipart strict fields; empty-text policy pending; non-idempotent
E-04	GET	/api/v1/listings/{listing_id}	Static contract only
E-05	POST	/api/v1/listings/{listing_id}/analyses	New run; persist failed status and retry
E-06	GET	/api/v1/listings/{listing_id}/analyses	Static contract only
E-07	GET	/api/v1/analyses/{analysis_run_id}	Static contract only
E-08	POST	/api/v1/suggestions/{suggestion_id}/decision	One immutable decision per suggestion; conflict on repeat
E-09	GET	/api/v1/review-queue	Choose latest completed/partial run before queue filter
E-10	GET	/api/v1/listings/{listing_id}/image	Binary image not JSON

10. UI/UX Integration Validation

Member 5 explicitly calls prototype.html standalone, hard-coded and not connected to models, FastAPI, or persistence (M5 p.1,3; HTML footer line 279). Its fixed candidate category options are provisional (HTML line 176). The staged progress bar has no matching live progress field in the synchronous API (HTML lines 206–210; M4 Section 3). Recommended UI contract: source category list from E-02, use indeterminate busy status until backed by a real progress API, retain input validation context, and label simulation.

11. Requirements Traceability Audit

The original, independently issued FRS was not supplied. The 25 titles/statements and priorities used here are reproduced from Member 2 SmartListing_FR_Traceability_Matrix.csv, treated as secondary evidence only. Every FR has mapped design references and planned tests, but this is not a verified signed-FRS coverage percentage. FULL implemented coverage and runtime pass percentages are intentionally not calculated.

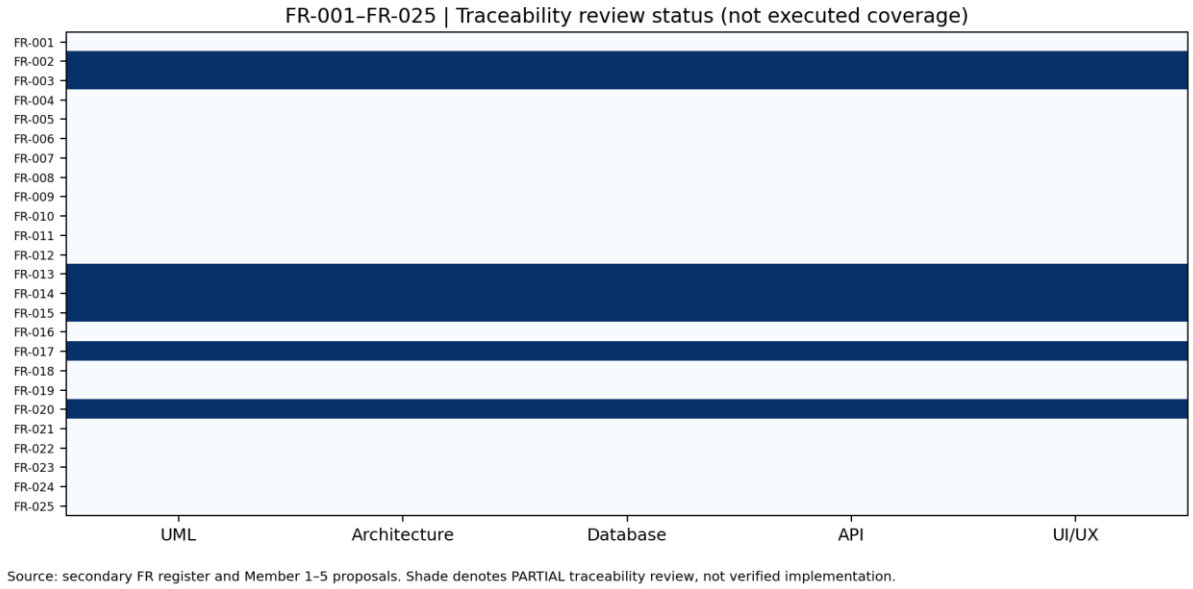
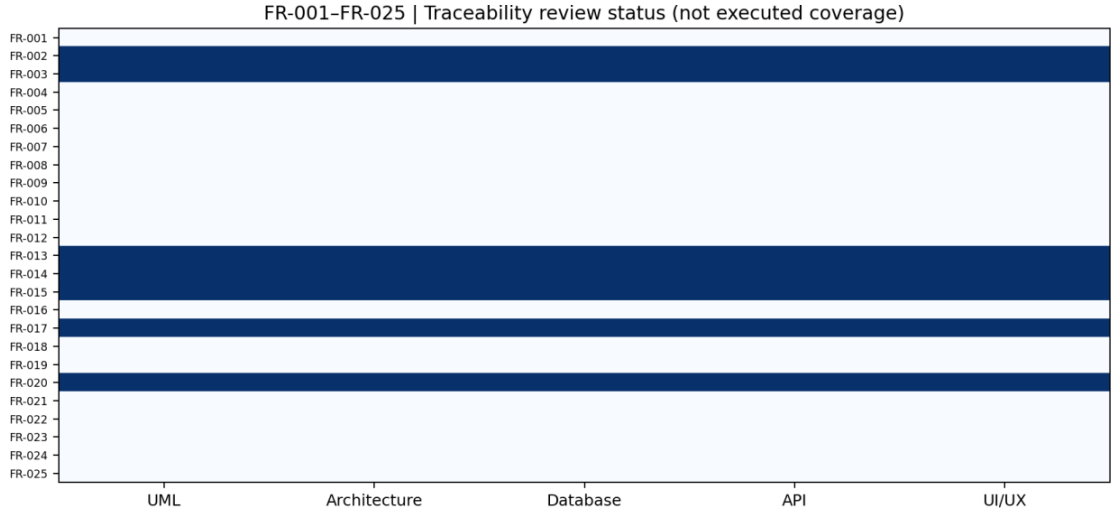


Figure B. Requirements coverage visualization — design-level only

Traceability review labels across 25 secondary FR entries: {'NOT VERIFIED': 18, 'PARTIAL': 7}. Denominator = 25 transcribed FR rows; numerator is document-level review classification, not confirmed requirement fulfillment.



Source: secondary FR register and Member 1–5 proposals. Shade denotes PARTIAL traceability review, not verified implementation.

12. State and Workflow Verification

State / event	Proposed guard / effect	Evidence
running	queue_state null	SQL ck_analysis_run_08
completed	queue_state required	SQL ck_analysis_run_08
partial	needs_review queue	SQL ck_analysis_run_08
failed	queue_state null; excluded by E-09	SQL; M4 E-09
accept / edit / ignore	One seller_decision per suggestion; edit requires final text	SQL seller_decision constraints; M4 E-08
re-analysis	New analysis_run, retain prior runs	M4 E-05
publish	Not an MVP transition	M3 p.1; M4 p.2

Unresolved guard decisions include transaction boundaries for interrupted pipelines, repeated submit behavior and last-usable-run tie handling. No undocumented new lifecycle state is introduced.

13. Error Handling and Edge-Case Analysis

Planned negative routes include missing/invalid/corrupt/oversized image, empty title/description, unsupported category, malformed IDs, missing resources, duplicate decisions, timeouts, CV/NLP errors, DB outages, absent prediction and conflicting simultaneous findings. API error envelope and UI recovery must be matched with a running server and recorded as acceptance evidence.

14. Comprehensive Design-Level Test Plan

Purpose: ensure approved design contracts can be implemented and verified without changing scope. Levels: static schema checks, module contract tests, API integration tests, stateful persistence tests, UI journey walkthrough and NFR tests. Execution evidence is not supplied. Every test is PLANNED. Numeric acceptance limits are deliberately deferred until NFR approval. Fixtures must remain synthetic/illustrative and labeled as such.

15. Detailed Test Case Catalogue

41 numbered design-level test scenarios are fully specified in the companion workbook (Test Cases tab). Selected core scenarios follow.

ID	Objective	Expected result
TC-001	Valid image and fields	Created listing, new analysis and result
TC-002	Type prediction	Product type prediction returns selected source and confidence
TC-003	Color prediction	Color evidence returned
TC-004	Blur flag	IMAGE_BLUR reason with no mismatch severity
TC-005	NLP claims	Claims and script route correctly identified
TC-006	Type mismatch	TYPE_MISMATCH with permitted severity
TC-007	Color mismatch	COLOR_MISMATCH with family rule
TC-008	Missing attribute	MISSING_ATTRIBUTE review flag
TC-009	Low confidence	LOW_CONFIDENCE; needs_review
TC-010	Unsupported category	UNSUPPORTED_CATEGORY and no forced definitive type
TC-011	Reason code mapping	Recognized reason codes; nullable flag severity
TC-012	Scoring	0–100 or NULL optional quality score and level
TC-013	Review queue	Latest usable run selected before filtering
TC-014	Template fix	Bounded suggestion and before/after
TC-015	Accept decision	Immutable accept record, no publication
TC-016	Edit decision	Edit payload saved and original retained
TC-017	Ignore decision	Ignore recorded, original retained

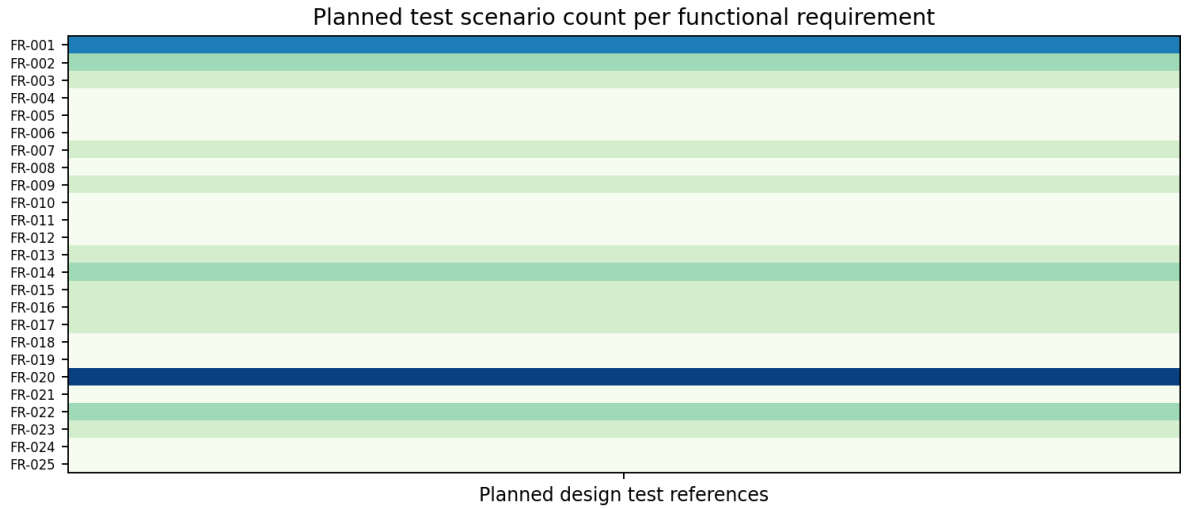
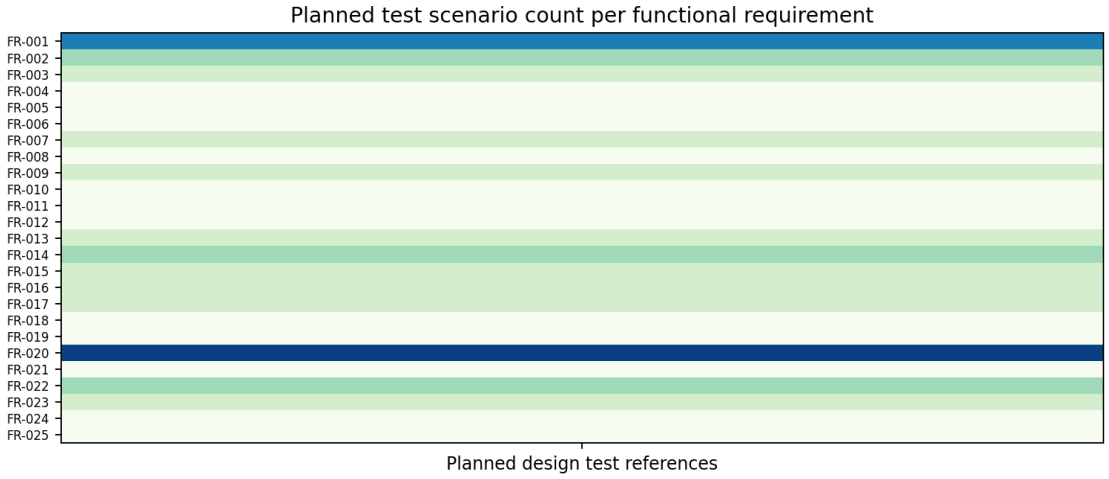


Figure E. Planned test coverage distribution



16. Risk and Defect Register

Finding counts are calculated directly from this review register: {'Major': 9, 'Minor': 2, 'Observation': 3}. Severity uses Critical = impossible core flow; Major = material integration/blocker; Minor = limited documentation/usability; Observation = unresolved choice or recommendation, not proven defect.

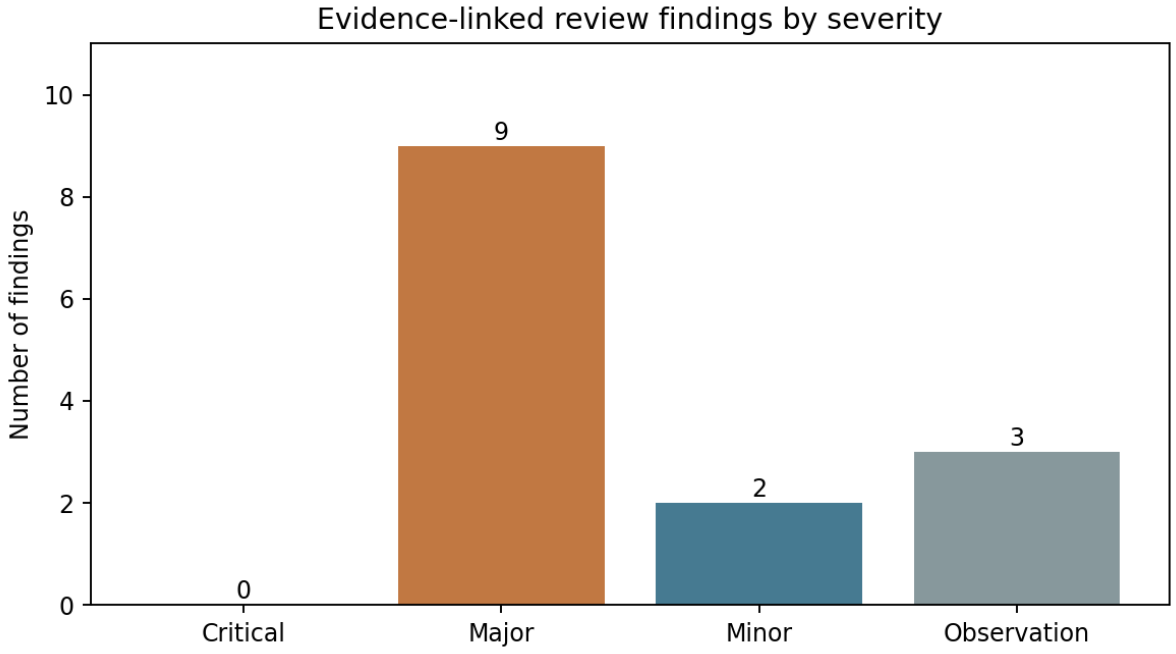
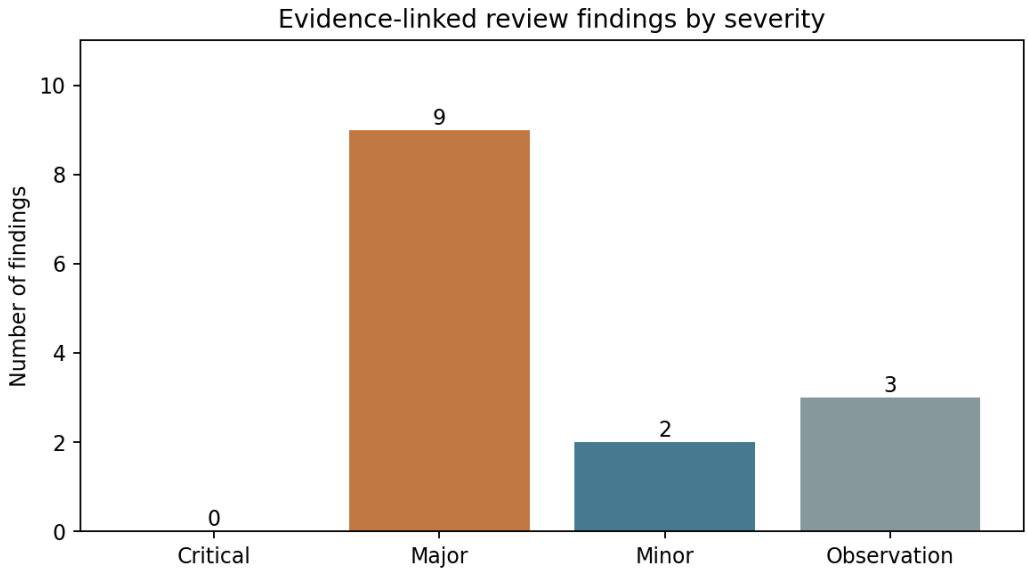


Figure C. Recorded design risks by category



F-01 | Major | Finding severity serialization

Observed: API returns Critical/Major/Minor while SQL CHECK allows lowercase critical/major/minor.

Evidence: M4 PDF p.3,p.13 (Section 7); SQL/01_create_schema.sql ck_analysis_finding_03

Owner: M2 + M4 Action: Define canonical API/DB mapping and implement serialization tests

Closure: Signed enum map and API payload-to-SQL fixture

F-02 | Major | Progress reporting contract

Observed: HTML staged percent progress has no stage-by-stage response in synchronous API; API acknowledges discrepancy and prescribes indeterminate indicator.

Evidence: prototype(1).html lines 206–210, 319; M4 PDF pp.3, approx.27 Section 11

Owner: M4 + M5 Action: Implement indeterminate waiting indicator or approve async progress API

Closure: Updated prototype and documented actual progress source

F-03 | Major | Five-category taxonomy not frozen

Observed: SQL seed intentionally empty and UI ships five illustrative options; live API categories cannot support selection until EDA approval.

Evidence: SQL/03_seed_reference_data.sql; M5 PDF p.3; prototype(1).html line 176; M4 E-02

Owner: M1 + M2 + M5 Action: Complete EDA, freeze category codes, bind UI options to E-02

Closure: Signed class vocabulary, seed and category API contract

F-04 | Major | Input empty-text policy

Observed: UI requires title and description, but underlying FRS notes empty-text behavior as unresolved; API specifies rejection as a proposed choice.

Evidence: M5 PDF p.3 (Section 1); prototype(1).html line 336; M4 PDF Section 13 A-03

Owner: Requirements owner + M4 + M5 Action: Approve empty-text behavior and align validation/tests

Closure: Approved decision + API/UI matching behavior

F-05 | Major | PostgreSQL run evidence

Observed: Validation report states 140 author-reported checks and SQLite translation; production PostgreSQL DDL not run.

Evidence: db/Validation_Report.md Summary; M2 PDF p.1

Owner: M2 Action: Execute all SQL scripts and boundary tests on isolated PostgreSQL instance

Closure: PostgreSQL execution log and assertions

F-06 | Major | No running API verification

Observed: API describes 10 endpoints as proposed; no server, real API results or contract tests provided.

Evidence: M4 PDF p.1 design boundary

Owner: M4 Action: Implement FastAPI stubs and validate schema, errors, transactions

Closure: Contract suite results on running API

F-07 | Minor | Milestone dates require reconciliation

Observed: UML cover identifies M1 6 Nov 2026 whereas handoff target 30 Oct 2026; could be internal vs academic milestone.

Evidence: M1 PDF p.1; Member 6 user-supplied assignment prompt Section 11A

Owner: Team leader Action: Clarify team review vs university milestone; preserve both if distinct

Closure: Approved schedule note

F-08 | Observation | Prototype boundary appropriately disclosed

Observed: Standalone HTML says hard-coded scenario outputs; architecture separately states Streamlit target. Not evidence of misrepresentation.

Evidence: M5 PDF pp.1,3; prototype(1).html line 279; M3 PDF p.1

Owner: M3 + M5 Action: Maintain explicit design-vs-build label during demo

Closure: Presentation footnote retained

F-09 | Major | Retry, partial success, queue rule

Observed: API E-09 selects latest completed/partial run, excluding newer failed runs. SQL permits failed with NULL queue; ensure every design uses the same rule.

Evidence: M4 PDF Section 5.9 E-09; SQL/01_create_schema.sql ck_analysis_run_08; SQL/04_validation_queries.sql

Owner: M1 + M2 + M4 Action: Review latest-usable-run query and failed retry acceptance tests

Closure: Run fixtures for completed, partial and failed ordering

F-10 | Major | Failure persistence transaction semantics

Observed: API proposes committing listing/running run before inference failure and retaining a failed run for retry; confirm rollback/compensation for partial child writes.

Evidence: M4 PDF E-03/Section 11 database mapping; SQL analysis_run constraints

Owner: M2 + M4 Action: Define commit boundaries and retry replay policy

Closure: Sequence + DB integration tests

F-11 | Observation | Requirements authority

Observed: FR-001..025 titles/statements can be transcribed from Member 2 CSV but original signed FRS not separately available in uploads.

Evidence: db/SmartListing_FR_Traceability_Matrix.csv; M4 PDF p.1

Owner: Team leader Action: Provide FRS and perform line-by-line authoritative reconciliation

Closure: Original FRS and trace sign-off

F-12 | Major | Idempotency of submit and decision

Observed: E-03 creates new listing on retry, E-08 limits to one decision; network retries can duplicate submissions or surface conflict.

Evidence: M4 PDF p.3 idempotency; M4 E-03/E-08

Owner: M4 + M2 Action: Specify client retry semantics, uniqueness, and HTTP 409 behavior

Closure: Duplicate-post tests and agreed API policy

F-13 | Minor | No authentication deployment guard

Observed: MVP explicitly omits accounts; API says local network exposure only. Deployment constraints need documented enforcement.

Evidence: M4 PDF p.2–3 authentication; M2 PDF p.1 design boundary

Owner: M3 + M4 Action: Specify localhost binding and access control at deployment edge

Closure: Deployment config and local-only smoke test

F-14 | Observation | Acceptance thresholds unresolved

Observed: Model thresholds, performance goals and category EDA are not approved; final quantitative NFR pass gates cannot be claimed.

Evidence: M1 PDF p.3 evidence discipline; M5 PDF p.3; M4 Section 13

Owner: Team leader + M3 Action: Freeze acceptance criteria after evaluation planning

Closure: Approved NFR/threshold baseline

17. Open Decisions and Assumptions

ID	Decision	Owner	Deadline / trigger
D-01	Milestone chronology	Team leader	30 Oct 2026
D-02	Approved five categories	Team leader / CV lead	Before API integration
D-03	Severity representation	M2 + M4	Before API integration
D-04	Empty text policy	Requirements owner	Before UI/API implementation
D-05	Progress UX	M4 + M5	Before UI integration
D-06	Commit / retry behavior	M2 + M4	Before integration tests
D-07	Queue definition	M1 + M2 + M4	Before queue integration
D-08	Duplicate submission policy	M4	Before integration tests
D-09	Acceptance limits and thresholds	Requirements owner / M3	Before model validation
D-10	PostgreSQL deployment	M2 + M3	Before DB execution

Proposed deadlines mean relative project gates unless the team confirms a calendar date. The class vocabulary, numeric thresholds, technology selection, and stakeholder sign-offs remain open.

18. Technical Readiness Assessment

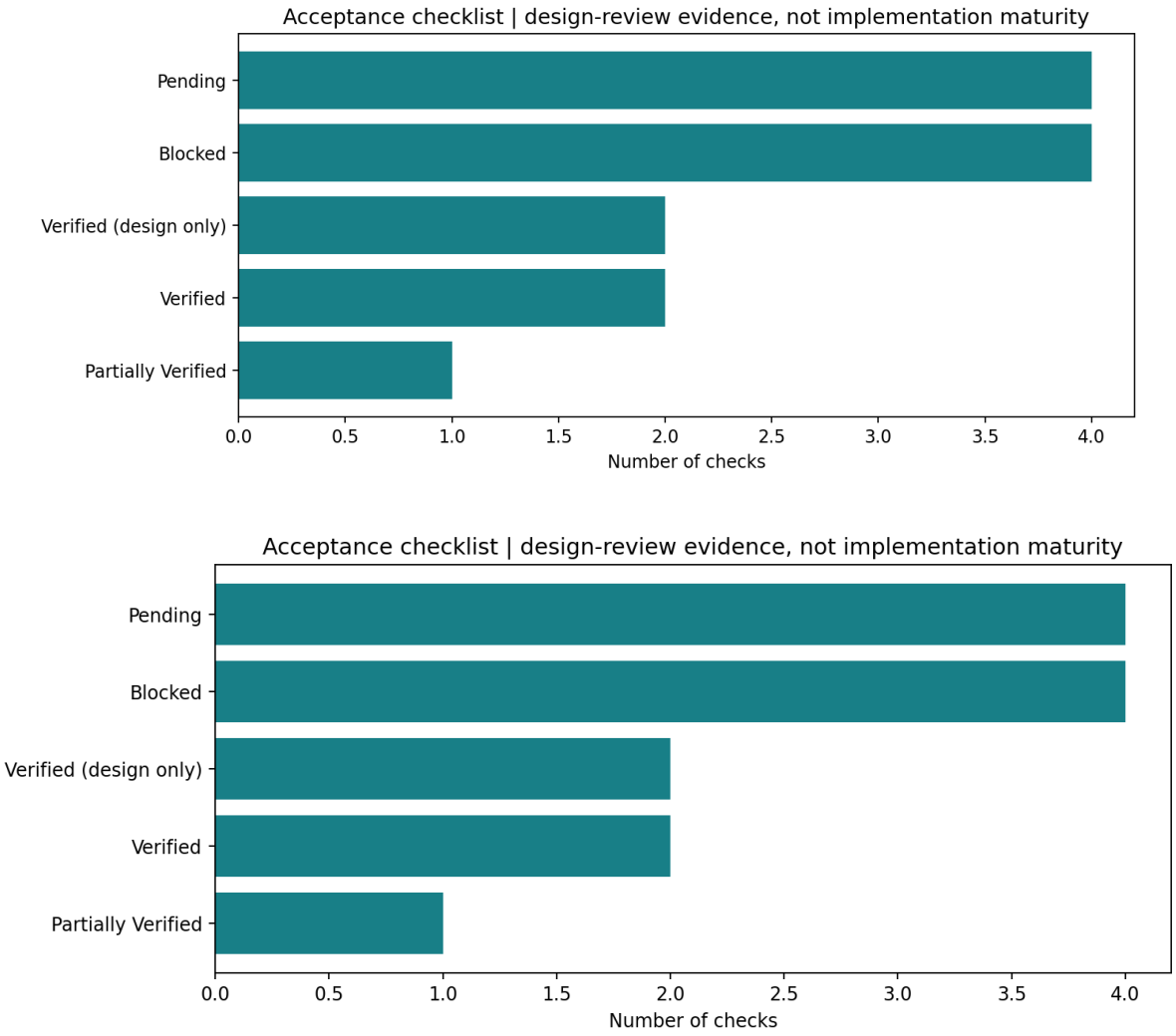


Figure D. Evidence-based design readiness

Verdict: CONDITIONALLY READY for contract-led implementation, not ready for integrated acceptance. This classification is qualitative, based on evidence, not an invented weighted maturity percentage. Verification is blocked by missing original FRS/rubric and absent executed PostgreSQL/API tests.

19. Corrective Action Plan

Suggested order: (1) obtain authoritative FRS and milestone explanation; (2) freeze five EDA categories and input/threshold policy; (3) resolve severity enum, queue and transaction/retry semantics with signed contracts; (4) update API, DB, UML and UI handoffs; (5) execute PostgreSQL migrations and edge cases; (6) implement FastAPI and Streamlit adapters and run contract/UI tests; (7) re-run traceability, produce approval record. See Corrective_Action_Plan.md.

20. Final Design Acceptance Checklist

ID	Criterion	Review status
C01	Scope: 5 EDA classes; type/color only	Partially Verified
C02	Canonical components and boundaries	Verified

ID	Criterion	Review status
C03	10 API endpoint specifications	Verified (design only)
C04	API and database severity case mapping	Blocked
C05	PostgreSQL DDL integration	Blocked
C06	Running FastAPI contract tests	Blocked
C07	Prototype disclosed as simulated	Verified
C08	Input policy team-approved	Pending
C09	Queue latest usable run verified in DB	Pending
C10	Original FRS independently verified	Blocked
C11	Design milestone dates reconciled	Pending
C12	Seller decisions never publish	Verified (design only)
C13	Numeric NFR acceptance thresholds approved	Pending

21. Review Conclusion

The Member 1–5 proposals provide a substantive design basis and appropriately avoid claiming real model training or production integration. The primary identified integration issue is severity casing; substantive gates remain category selection, policy approvals, transient progress, lifecycle failure and real Postgres/API evidence. This Member 6 work is an independent design assurance handoff. It is not an acceptance signature and not a completed implementation.

22. Appendices and Evidence References

Workbook tabs: Source Register, Consistency Matrix, Requirements Traceability, API Contract Audit, Database Validation, Test Cases, Risk Register, Open Decisions, Final Acceptance Checklist. Exhibit SVG/PNG assets are original interpretations of inspected artifacts. Original references: M1 analysis PDF/DOCX; M2 design PDF and ZIP SQL/Validation_Report/trace CSV; M3 architecture PDF; M4 API PDF; M5 UX PDF and prototype.html. The source register includes document-level provenance and limitations.

PART II — COMPREHENSIVE ENGINEERING REFERENCE

Expanded technical reference, contract-by-contract verification handbook, and implementation acceptance guide | Member 6 | Version 2.0 | 30 October 2026

Evidence policy: descriptions of intended behavior are proposals unless explicitly evidenced by a source. Planned tests are not execution results. No fabricated category labels, numeric model accuracy, stakeholder sign-offs, or PostgreSQL runtime outcomes are presented.

1. Governance, provenance and source authority

Responsible reviewers: Member 6 / team lead

Evidence basis: M1 UML PDF pp.1–3; M2 Database PDF pp.1–3; M3 Architecture PDF p.1; M4 API PDF pp.1–3; M5 UX PDF pp.1–3

A. Technical design and operational interpretation

1. 01. Make an explicit distinction between official requirements, revised scope, member-authored proposals and executable proof. The member design packages do not replace the source FRS or official marking rubric when those originals are absent.
2. 02. Track each proposed interpretation as either confirmed by an authoritative source, second-hand via a member package, or open pending sign-off.
3. 03. The M1 PDF and DOCX differ in pagination (84 versus 80 pages). Preserve both as separate representations until their text and diagrams can be reconciled; page references must name the format.

B. Verification protocol and edge cases

Check 1 — Verify source identifiers, page number, date, version, responsible member and approval status

Procedure: Verify source identifiers, page number, date, version, responsible member and approval status. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Check that duplicate titles or archived copies do not replace the selected source without an explicit change note

Procedure: Check that duplicate titles or archived copies do not replace the selected source without an explicit change note. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Escalate missing source FRS and rubric as verification limitations, not assumed compliance

Procedure: Escalate missing source FRS and rubric as verification limitations, not assumed compliance. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Signed source register — assigned to Member 6 / team lead. Status: proposed/pending verification unless supplied as executable evidence.
- Canonical file manifest and hashes — assigned to Member 6 / team lead. Status: proposed/pending verification unless supplied as executable evidence.
- Version-difference review — assigned to Member 6 / team lead. Status: proposed/pending verification unless supplied as executable evidence.
- Authority and limitation log — assigned to Member 6 / team lead. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

2. Scope constraints and non-goals

Responsible reviewers: Lead analyst / team lead

Evidence basis: M3 p.1; M5 pp.1–3; M4 pp.1–2

A. Technical design and operational interpretation

4. 01. The MVP is a single fashion listing image with seller-provided title, description and category; its analytic domain is Product Type and Color only.
5. 02. Five final product categories are to be frozen after EDA. UI candidate labels and illustrative prediction examples do not constitute an approved class taxonomy.
6. 03. The dataset supplies images and metadata but no natural listing description field; manually or synthetically written descriptions must be explicitly labeled as evaluation inputs.
7. 04. The pipeline proposes CV and NLP evidence, deterministic rule comparison, triage and template suggestions. The seller retains Accept, Edit and Ignore control; no marketplace publishing.
8. 05. Do not introduce user accounts, payment, pricing, size/material inference, live marketplaces, bulk-processing or LLM-based core classifications into the MVP just to make the documentation sound richer.

B. Verification protocol and edge cases

Check 1 — Compare every screen, SQL column, API schema and UML node against this boundary

Procedure: Compare every screen, SQL column, API schema and UML node against this boundary. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Check for accidental mention of extra attributes, autonomous publication or unsupported AI components

Procedure: Check for accidental mention of extra attributes, autonomous publication or unsupported AI components. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Freeze class names after documented EDA and update references centrally

Procedure: Freeze class names after documented EDA and update references centrally. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Scope approval note — assigned to Lead analyst / team lead. Status: proposed/pending verification unless supplied as executable evidence.
- Class-freeze decision — assigned to Lead analyst / team lead. Status: proposed/pending verification unless supplied as executable evidence.
- Data provenance declaration — assigned to Lead analyst / team lead. Status: proposed/pending verification unless supplied as executable evidence.
- Change-request record — assigned to Lead analyst / team lead. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

3. End-to-end analysis journey

A. Technical design and operational interpretation

9. 01. Seller uploads one locally validated image and enters title, description and category. The client submits the data through the API rather than reading the database.
10. 02. The FastAPI Backend validates the request, creates a listing snapshot, orchestrates parallel CV and NLP processing, then passes typed outputs to mismatch detection.
11. 03. The Scoring & Triage module computes listing-level status from findings and confidence. The Suggestion module generates a bounded before/after proposal only where applicable.
12. 04. The backend stores the input snapshot, analysis evidence, findings and any suggestion. The UI presents a traceable result and receives a separate explicit decision.
13. 05. An analysis failure must not be displayed as a valid no-mismatch result. Processing outcome and semantic inconsistency are independent concepts.

B. Verification protocol and edge cases

Check 1 — Trace an image through every component name and type boundary

Procedure: Trace an image through every component name and type boundary. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Trace no-claim, low-confidence, contradiction and partial processing branches

Procedure: Trace no-claim, low-confidence, contradiction and partial processing branches. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Confirm all visible decisions have corresponding persisted records and statuses

Procedure: Confirm all visible decisions have corresponding persisted records and statuses. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Annotated sequence walkthrough — assigned to M1 + M3 + M4 + M5. Status: proposed/pending verification unless supplied as executable evidence.
- Input/output map — assigned to M1 + M3 + M4 + M5. Status: proposed/pending verification unless supplied as executable evidence.
- Partial-failure behavior decision — assigned to M1 + M3 + M4 + M5. Status: proposed/pending verification unless supplied as executable evidence.
- Acceptance screenshots from a real app, when available — assigned to M1 + M3 + M4 + M5. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

4. Component contract and responsibility allocation

Responsible reviewers: M3 / API implementer

Evidence basis: M3 pp.1–4

A. Technical design and operational interpretation

14. 01. Streamlit Frontend handles seller-facing presentation, client validation and decision entry; it should not run model inference or make direct database modifications.
15. 02. FastAPI Backend owns validation, orchestration, error handling and persistence boundaries.
16. 03. CV Module returns image-backed product type/color evidence and associated confidence/uncertainty representation.
17. 04. NLP Module extracts supported product type/color claims from user text, including bounded Arabic lexicon behavior if approved.
18. 05. Mismatch Detection Module produces reason-coded, attributed findings. Scoring & Triage Module derives listing-level decisions. Suggestion Module produces controlled proposals, not published edits.
19. 06. Database stores relational records; Model & Lexicon Store retains weights, configuration and vocabulary separately. Shared Schemas are imported consistently.

B. Verification protocol and edge cases

Check 1 — Inspect every dependency edge and reject direct UI-to-database coupling

Procedure: Inspect every dependency edge and reject direct UI-to-database coupling. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Document typed inputs/outputs and nullable fields for each module

Procedure: Document typed inputs/outputs and nullable fields for each module. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Use module stubs in contract tests before availability of trained models

Procedure: Use module stubs in contract tests before availability of trained models. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Component/interface catalogue — assigned to M3 / API implementer. Status: proposed/pending verification unless supplied as executable evidence.
- Dependency diagram — assigned to M3 / API implementer. Status: proposed/pending verification unless supplied as executable evidence.
- Stub contract fixtures — assigned to M3 / API implementer. Status: proposed/pending verification unless supplied as executable evidence.
- Responsibility acceptance checklist — assigned to M3 / API implementer. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

5. Image and text intake validation

Responsible reviewers: Frontend + API owner

Evidence basis: M4 pp.2–3 and validation section; M5 pp.3–5

A. Technical design and operational interpretation

20. 01. A listing submission must include the fields required by the approved requirement baseline; member proposals currently assume image, title, description and category.
21. 02. Image MIME type, content decoding, maximum size, duplicate submissions, prohibited file extensions and Unicode whitespace behavior must be agreed before implementation acceptance.
22. 03. Client-side validation is advisory; FastAPI must independently enforce validated formats, length limits and supported category membership.
23. 04. Do not equate a field missing in JSON with null or an empty string: each case should have intentional, documented semantics.
24. 05. Client errors must preserve entered text and explain which specific input requires correction without presenting them as model findings.

B. Verification protocol and edge cases

Check 1 — Test valid supported image, corrupt image, spoofed MIME, empty image and oversized image

Procedure: Test valid supported image, corrupt image, spoofed MIME, empty image and oversized image. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Test missing, blank, long, Arabic and mixed-script title and description

Procedure: Test missing, blank, long, Arabic and mixed-script title and description. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Test category set unavailable, unknown category and stale selection

Procedure: Test category set unavailable, unknown category and stale selection. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Validation decision table — assigned to Frontend + API owner. Status: proposed/pending verification unless supplied as executable evidence.
- Negative request examples — assigned to Frontend + API owner. Status: proposed/pending verification unless supplied as executable evidence.
- Field-level error copy — assigned to Frontend + API owner. Status: proposed/pending verification unless supplied as executable evidence.
- Signed size/format limits — assigned to Frontend + API owner. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

6. CV inference evidence and uncertainty

Responsible reviewers: CV owner / test lead

Evidence basis: M3 pp.1–4; M1 architecture and design diagrams

A. Technical design and operational interpretation

25. 01. Product type classification uses one of five EDA-selected classes, not arbitrary open-vocabulary labels, in the proposed deployed contract.
26. 02. Color evidence is a separate supported attribute. Never imply that model confidence proves semantic correctness or calibrated reliability without measured evaluation.
27. 03. Record model version and prediction provenance when available. Proposed thresholds remain configurable and explicitly unapproved until experiments establish them.
28. 04. Where top class cannot be relied upon, preserve an uncertainty or abstention path rather than inventing a definitive label.
29. 05. The design describes model boundaries; this Member 6 document does not claim model weights, trained metrics, calibrated ECE or successful inference tests.

B. Verification protocol and edge cases

Check 1 — Check class-code compatibility with categories in API and database

Procedure: Check class-code compatibility with categories in API and database. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Test malformed model outputs, unknown class, invalid probabilities and missing color

Procedure: Test malformed model outputs, unknown class, invalid probabilities and missing color. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Confirm calibrated and uncalibrated confidence terminology is not conflated

Procedure: Confirm calibrated and uncalibrated confidence terminology is not conflated. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Model output JSON schema — assigned to CV owner / test lead. Status: proposed/pending verification unless supplied as executable evidence.
- EDA class dictionary — assigned to CV owner / test lead. Status: proposed/pending verification unless supplied as executable evidence.
- Confidence/abstention policy — assigned to CV owner / test lead. Status: proposed/pending verification unless supplied as executable evidence.
- Real model evaluation report, when produced — assigned to CV owner / test lead. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

7. NLP claim extraction and multilingual constraints

Responsible reviewers: NLP owner / QA

Evidence basis: M3 pp.1–4; M5 p.3

A. Technical design and operational interpretation

30. 01. Parse seller-generated or synthetic listing text for Product Type and Color claims only; store claim provenance to distinguish title from description and synthesized test input.
31. 02. No inferred field should be treated as a seller-stated fact unless the text actually contains a supported claim.
32. 03. A fixed Arabic lexicon is a bounded design assumption; do not claim broad Arabic understanding or a bilingual end-to-end UI unless implementation proves it.
33. 04. Multiple/conflicting claims and absent claims require explicit deterministic rules. A no-claim condition is not necessarily a mismatch.
34. 05. Returned claim spans, normalized values and confidence information must match database structure and API serialization choices.

B. Verification protocol and edge cases

Check 1 — Validate title-only, description-only and contradictory text

Procedure: Validate title-only, description-only and contradictory text. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Check Arabic script, case folding, synonyms, mixed English/Arabic and unsupported vocabulary

Procedure: Check Arabic script, case folding, synonyms, mixed English/Arabic and unsupported vocabulary. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Verify that missing claims cannot silently become a false positive

Procedure: Verify that missing claims cannot silently become a false positive. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Claim taxonomy and lexicon version — assigned to NLP owner / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Text fixture suite — assigned to NLP owner / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Normalization policy — assigned to NLP owner / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Provenance audit log — assigned to NLP owner / QA. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

8. Mismatch detection and reason-code contract

Responsible reviewers: Mismatch module owner

Evidence basis: M3 pp.1–4; M4 sections 6–8; M5 sections 12–13

A. Technical design and operational interpretation

35. 01. Compare only supported claim/evidence pairs. A type discrepancy and color discrepancy should remain separately attributable to the triggering attribute.
36. 02. Reason code, severity, and human-readable explanation must be deterministic and shared between persistence, HTTP and UI.
37. 03. Duplicate warnings must be constrained as documented, and inconsistent evidence should be presented in severity order rather than hidden by a single opaque score.
38. 04. Near-shade or color-family rules and uncertainty behavior must not be silently improvised by Member 6.
39. 05. The observed design mismatch between API title-case severity and SQL lowercase CHECK is a contract inconsistency requiring explicit mapping or schema change before acceptance.

B. Verification protocol and edge cases

Check 1 — Exercise exact matches, type mismatches, color mismatches, dual mismatches and unknown claims

Procedure: Exercise exact matches, type mismatches, color mismatches, dual mismatches and unknown claims. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Assert per-attribute reason-code uniqueness where the source requires it

Procedure: Assert per-attribute reason-code uniqueness where the source requires it. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Round-trip each severity between SQL, Pydantic and UI

Procedure: Round-trip each severity between SQL, Pydantic and UI. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Canonical enum table — assigned to Mismatch module owner. Status: proposed/pending verification unless supplied as executable evidence.
- Rule decision table — assigned to Mismatch module owner. Status: proposed/pending verification unless supplied as executable evidence.
- Negative/ambiguity fixtures — assigned to Mismatch module owner. Status: proposed/pending verification unless supplied as executable evidence.
- Severity mapping decision — assigned to Mismatch module owner. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

9. Scoring, triage and Review Queue state machine

Responsible reviewers: Scoring owner / API owner

Evidence basis: M3 pp.1–4; M4 sections 5–7; M2 sections on lifecycle

A. Technical design and operational interpretation

40. 01. Score, inconsistency level and queue status are different concepts: a numeric listing score does not automatically prove the queue state or decision.
41. 02. The Review Queue must define a source-of-truth rule for choosing the latest usable analysis run; M4 documents latest completed/partial handling.
42. 03. Failed/partial runs must not accidentally replace usable historical results or imply that the seller accepted a suggestion.
43. 04. The state machine should make legal transitions explicit; conflict responses must not mutate persisted decisions.
44. 05. Eligibility to display a queue item, and who sees it, must align with the no-user-accounts MVP boundary.

B. Verification protocol and edge cases

Check 1 — Create runs in sequence: complete→failed, complete→partial, partial→complete

Procedure: Create runs in sequence: complete→failed, complete→partial, partial→complete. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Verify queue sort and tie rules, pagination and missing records

Procedure: Verify queue sort and tie rules, pagination and missing records. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Ensure repeated processing does not overwrite history

Procedure: Ensure repeated processing does not overwrite history. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- State-transition table — assigned to Scoring owner / API owner. Status: proposed/pending verification unless supplied as executable evidence.
- Queue selection SQL and test fixtures — assigned to Scoring owner / API owner. Status: proposed/pending verification unless supplied as executable evidence.
- Conflict behavior contract — assigned to Scoring owner / API owner. Status: proposed/pending verification unless supplied as executable evidence.
- Concurrency acceptance record — assigned to Scoring owner / API owner. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

10. Controlled suggestion and seller decision semantics

Responsible reviewers: Suggestion owner / persistence owner

Evidence basis: M2 data model; M3 p.1; M4 pp.2–3; M5 UX journey

A. Technical design and operational interpretation

45. 01. Suggestions are grounded, bounded text proposals only; they should not add size, material, price or unsupported product facts.
46. 02. The UI must show original and proposed text separately with reasons, and the seller must choose Accept, Edit or Ignore explicitly.
47. 03. The data model allows an optional suggestion and at most one decision per suggestion. Do not present Accept as an external publish operation.
48. 04. Edit must have a documented stored final text or decision-payload representation, not merely a visual button.
49. 05. Decision attempts must be conflict-safe under double click and simultaneous requests.

B. Verification protocol and edge cases

Check 1 — Test zero suggestion, single suggestion, all three decisions and user-edited content

Procedure: Test zero suggestion, single suggestion, all three decisions and user-edited content. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Test duplicate decision and invalid action

Procedure: Test duplicate decision and invalid action. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Verify no hidden side effect or marketplace write occurs

Procedure: Verify no hidden side effect or marketplace write occurs. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Decision enum contract — assigned to Suggestion owner / persistence owner. Status: proposed/pending verification unless supplied as executable evidence.
- Before/after examples — assigned to Suggestion owner / persistence owner. Status: proposed/pending verification unless supplied as executable evidence.
- Persistence proof — assigned to Suggestion owner / persistence owner. Status: proposed/pending verification unless supplied as executable evidence.
- No-publishing negative assurance — assigned to Suggestion owner / persistence owner. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

11. Relational data architecture and integrity

Responsible reviewers: Database designer / QA

Evidence basis: M2 pp.1–3; schema SQL ZIP

A. Technical design and operational interpretation

50. 01. The proposed eight tables are supported_category, product_listing, analysis_run, visual_prediction, text_claim, analysis_finding, suggested_correction and seller_decision.
51. 02. The database package describes eight tables, 64 columns, seven foreign keys and guarded indexes/constraints. Those are document-level reported quantities, not PostgreSQL runtime proof.
52. 03. A listing snapshot should remain traceable to its source and should not silently change when analysis is repeated.
53. 04. Separate analysis runs, findings, claims, predictions and decisions enable auditability; relational keys must preserve lineage.
54. 05. The supplied schema archive includes create, constraint/index, seed and validation SQL. SQLite portability/static checks do not prove PostgreSQL executes the intended DDL unchanged.

B. Verification protocol and edge cases

Check 1 — Execute PostgreSQL migrations from an empty schema and rerun for failure behavior

Procedure: Execute PostgreSQL migrations from an empty schema and rerun for failure behavior. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Exercise PK/FK/NOT NULL/UNIQUE/CHECK under valid and invalid values

Procedure: Exercise PK/FK/NOT NULL/UNIQUE/CHECK under valid and invalid values. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Confirm rollback, cascading effects and duplicate analysis behavior

Procedure: Confirm rollback, cascading effects and duplicate analysis behavior. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- PostgreSQL execution log — assigned to Database designer / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Schema/ERD equivalence sheet — assigned to Database designer / QA. Status: proposed/pending verification unless supplied as executable evidence.
- DDL validation output — assigned to Database designer / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Rollback and referential tests — assigned to Database designer / QA. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

12. API design conventions and lifecycle

Responsible reviewers: API owner / integration QA

Evidence basis: M4 PDF sections 3–5

A. Technical design and operational interpretation

55. 01. Base path is /api/v1. Responses use JSON; image submission uses multipart/form-data. Identifiers are SQL-derived integer values and timestamps are ISO 8601 UTC with Z.
56. 02. The API is local-only and has no account/session layer. The UI must not assume authentication or remote multi-user security not designed for this milestone.
57. 03. The proposed model is synchronous for POST analysis operations; it does not expose stage-by-stage progress.
58. 04. GET operations are idempotent; new submission and re-analysis create new entities; seller decisions are limited to one per suggestion.
59. 05. Missing versus null fields, request tracing, error envelope and stable enum serialization should receive implementation acceptance fixtures.

B. Verification protocol and edge cases

Check 1 — Test each method/path using valid and invalid IDs

Procedure: Test each method/path using valid and invalid IDs. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Verify 400/404/409/422/500 or documented alternatives against actual contract

Procedure: Verify 400/404/409/422/500 or documented alternatives against actual contract. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Validate response schema, UTC format and nullable keys

Procedure: Validate response schema, UTC format and nullable keys. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- OpenAPI export — assigned to API owner / integration QA. Status: proposed/pending verification unless supplied as executable evidence.
- Pydantic schemas — assigned to API owner / integration QA. Status: proposed/pending verification unless supplied as executable evidence.
- Contract test fixtures — assigned to API owner / integration QA. Status: proposed/pending verification unless supplied as executable evidence.
- HTTP capture archive — assigned to API owner / integration QA. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

13. UI/UX behaviors and truthful prototype status

Responsible reviewers: UX member / QA

Evidence basis: M5 pp.1–3 and sections 12–14; prototype.html

A. Technical design and operational interpretation

60. 01. The standalone prototype simulates proposed behavior using hard-coded demonstration outputs; it is not an implemented Streamlit application.
61. 02. Loading, empty, partial, no-mismatch, flagged, error and retry states should be described consistently with proposed API and persistence statuses.
62. 03. The category selector must eventually use a frozen EDA-backed list rather than illustrative candidates; an empty-category state needs deliberate UI copy.
63. 04. A percentage progress indicator cannot claim backend progress while the API is synchronous without stage telemetry. An indeterminate busy state is the accurate default.
64. 05. Responsive layout, keyboard focus, contrast and assistive error feedback need direct walkthrough evidence before claiming accessible deployment.

B. Verification protocol and edge cases

Check 1 — Verify forms and keyboard navigation at desktop/mobile

Procedure: Verify forms and keyboard navigation at desktop/mobile. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Check that every action has an API mapping or is clearly labeled demonstrative

Procedure: Check that every action has an API mapping or is clearly labeled demonstrative. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Compare visual reason/severity text and before/after suggestions with canonical enum policy

Procedure: Compare visual reason/severity text and before/after suggestions with canonical enum policy. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- UI state catalogue — assigned to UX member / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Screen-to-endpoint matrix — assigned to UX member / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Accessibility checklist — assigned to UX member / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Walkthrough notes — assigned to UX member / QA. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

14. Functional requirement and NFR traceability

Responsible reviewers: QA lead / requirements owner

Evidence basis: M2 traceability references; M1 section 11; M4 section 12

A. Technical design and operational interpretation

65. 01. The 25 FR identifiers are presently traceable through member-authored secondary material; original FRS was not supplied separately in this upload.
66. 02. Trace a requirement through analysis diagram, architecture component, relational entity, HTTP operation, UI state and acceptance scenario as applicable.
67. 03. Mark coverage as documentary, partial, contradicted or not verifiable. Do not convert design references into software test pass rates.
68. 04. NFR families include usability, reliability, security, maintainability, performance and technical constraints; exact benchmarks require approved source values.
69. 05. A traceability row is complete only when evidence is attributable and ambiguity has a named owner.

B. Verification protocol and edge cases

Check 1 — Verify all FR-001

Procedure: Verify all FR-001..FR-025 identifiers are present in the matrix. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Review missing API and UI mapping for each requirement

Procedure: Review missing API and UI mapping for each requirement. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Map every high-risk NFR to at least one planned measurable acceptance test

Procedure: Map every high-risk NFR to at least one planned measurable acceptance test. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- FR-to-design-to-test RTM — assigned to QA lead / requirements owner. Status: proposed/pending verification unless supplied as executable evidence.
- NFR acceptance matrix — assigned to QA lead / requirements owner. Status: proposed/pending verification unless supplied as executable evidence.
- Change-impact trace — assigned to QA lead / requirements owner. Status: proposed/pending verification unless supplied as executable evidence.
- Signed requirement baseline — assigned to QA lead / requirements owner. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

15. Quality strategy and execution levels

Responsible reviewers: QA lead / all members

Evidence basis: Member 6 original report sections 13–15

A. Technical design and operational interpretation

70. 01. Static review validates diagram coherence, contract names and schema constraints; it does not prove runtime execution.
71. 02. Unit tests should stub dependencies and verify mapping behavior. Integration tests should use an actual PostgreSQL database and FastAPI test client or server.
72. 03. End-to-end tests should exercise one user journey using a real implemented frontend, backend and persisted state; standalone HTML simulation is excluded.
73. 04. ML quality checks should use fixed train/validation/test splits and report genuine metrics only after executing reproducible evaluation.
74. 05. Release gating requires evidence links, timestamps, environment details and reproducible commands.

B. Verification protocol and edge cases

Check 1 — Store command, commit, environment, fixture and actual outcome for every test

Procedure: Store command, commit, environment, fixture and actual outcome for every test. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Separate observed failure from unexecuted/pending scenario

Procedure: Separate observed failure from unexecuted/pending scenario. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Run negative and recovery routes in addition to happy path

Procedure: Run negative and recovery routes in addition to happy path. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Test-plan workbook — assigned to QA lead / all members. Status: proposed/pending verification unless supplied as executable evidence.
- Execution evidence folder — assigned to QA lead / all members. Status: proposed/pending verification unless supplied as executable evidence.
- Defect ledger — assigned to QA lead / all members. Status: proposed/pending verification unless supplied as executable evidence.
- Release gate checklist — assigned to QA lead / all members. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

16. Cross-artifact defect management

Responsible reviewers: Integration lead

Evidence basis: Member 6 original finding register

A. Technical design and operational interpretation

75. 01. A finding has an identifier, exact conflicting elements, documentary evidence, severity, consequence, owner and proposed resolution.
76. 02. Proposals to fix discrepancies remain unapproved until the cross-member owner accepts the change and dependent artifacts are updated.
77. 03. Prioritize material contract incompatibilities such as severity serialization before downstream frontend implementation.
78. 04. Use status values OPEN, IN REVIEW, APPROVED FOR CHANGE, FIXED IN DESIGN, VERIFIED BY TEST and DEFERRED WITH RATIONALE.
79. 05. A closed design finding requires a rechecked source plus, where relevant, a new executable test.

B. Verification protocol and edge cases

Check 1 — Assign findings without inventing resolved outcomes

Procedure: Assign findings without inventing resolved outcomes. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Version each affected document and update change history

Procedure: Version each affected document and update change history. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Run targeted regression checks after contract changes

Procedure: Run targeted regression checks after contract changes. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Defect record — assigned to Integration lead. Status: proposed/pending verification unless supplied as executable evidence.
- Approval trail — assigned to Integration lead. Status: proposed/pending verification unless supplied as executable evidence.
- Changed artifact diff — assigned to Integration lead. Status: proposed/pending verification unless supplied as executable evidence.
- Regression evidence — assigned to Integration lead. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

17. Security, privacy and storage hygiene

Responsible reviewers: Backend / DBA / QA

Evidence basis: M2 security section; M4 conventions

A. Technical design and operational interpretation

80. 01. The MVP is designed for restricted local deployment; no user accounts are specified. A public network exposure assumption would contradict that control boundary.
81. 02. Avoid storing arbitrary client-supplied paths, exposing filesystem paths or returning model-store locations through HTTP.
82. 03. User image input must be decoded and validated safely; unsafe file and content handling are implementation risks, not documented successful exploits.
83. 04. Keep predictable data provenance and limit disclosure of uploaded images and listing content.
84. 05. Set retention and deletion behavior by team decision instead of assuming GDPR or production-grade retention automatically exists.

B. Verification protocol and edge cases

Check 1 — Attempt invalid path traversal and malformed image inputs during implementation tests

Procedure: Attempt invalid path traversal and malformed image inputs during implementation tests. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Verify response bodies do not leak secrets, local paths or debug traces

Procedure: Verify response bodies do not leak secrets, local paths or debug traces. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Confirm only intended network interfaces listen in local demonstration

Procedure: Confirm only intended network interfaces listen in local demonstration. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Threat review — assigned to Backend / DBA / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Secure-input test log — assigned to Backend / DBA / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Data retention decision — assigned to Backend / DBA / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Deployment configuration — assigned to Backend / DBA / QA. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

18. Performance, reliability and graceful degradation

Responsible reviewers: Backend / QA

Evidence basis: M1 NFR discussion; M4 API convention

A. Technical design and operational interpretation

85. 01. The synchronous POST request means a long inference call can block UI result delivery. No measured latency benchmark is available in these submitted design files.
86. 02. Timeouts, retries and partial analyses must have explicitly defined effects on stored analysis_run state.
87. 03. A CV outage and an NLP outage may produce different partial evidence; neither should display an authoritative clean result by default.
88. 04. Retry behavior must avoid inventing accidental duplicate seller decisions; submit/re-analysis creation behavior is intentionally non-idempotent.
89. 05. Performance thresholds should be approved before quantitative PASS/FAIL is assigned.

B. Verification protocol and edge cases

Check 1 — Inject model timeout, database unavailability and a interrupted request

Procedure: Inject model timeout, database unavailability and a interrupted request. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Capture actual latency distributions and memory usage before citing metrics

Procedure: Capture actual latency distributions and memory usage before citing metrics. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Check user-visible recovery state and old-run accessibility

Procedure: Check user-visible recovery state and old-run accessibility. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Failure matrix — assigned to Backend / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Timing measurement plan — assigned to Backend / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Recovery runbook — assigned to Backend / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Approved NFR thresholds — assigned to Backend / QA. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

19. Data provenance, dataset splits and reproducibility

Responsible reviewers: ML leads / QA

Evidence basis: M1 dataset baseline; M5 pp.1–3

A. Technical design and operational interpretation

90. 01. Fashion Product Images Small is the sole dataset context. Exact five categories require a reproducible post-EDA selection record.
91. 02. Product descriptions are not part of source styles.csv. Synthetic/manual descriptions must not be mislabeled as naturally observed retailer text.
92. 03. Leakage prevention requires stable image identifiers, deduplication rules, split definitions and deterministic random seeds.
93. 04. Proposed CV baselines and evaluation metrics must remain prospective until training is performed and artifacts are saved.
94. 05. Demonstration confidence values in diagrams, API examples and prototype are illustrative, not empirical metrics.

B. Verification protocol and edge cases

Check 1 — Hash source/split manifests and record dropped examples

Procedure: Hash source/split manifests and record dropped examples. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Check overlapping images or groups across train and test

Procedure: Check overlapping images or groups across train and test. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Identify every synthetic sentence in evaluation provenance

Procedure: Identify every synthetic sentence in evaluation provenance. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- EDA notebook and category map — assigned to ML leads / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Split manifest — assigned to ML leads / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Licensing review — assigned to ML leads / QA. Status: proposed/pending verification unless supplied as executable evidence.
- Experiment card — assigned to ML leads / QA. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

20. Release readiness and academic defense

Responsible reviewers: Team lead / Member 6

Evidence basis: Member 6 original readiness assessment

A. Technical design and operational interpretation

95. 01. The appropriate current state is conditionally ready for contract-first implementation, not accepted as working integrated software.
96. 02. Academic handoff can include reviewed design plans while explicitly stating unresolved points, document provenance and pending stakeholder approval.
97. 03. Before asserting production readiness, require PostgreSQL DDL execution, FastAPI contract tests, UI integration, real model evidence and defect closure.
98. 04. The member 1 document states a 6 November 2026 M1 milestone; the team handoff date referenced in this work is 30 October 2026. Reconcile against official course schedule.
99. 05. Preserve records of what Member 6 independently verified versus what other authors reported.

B. Verification protocol and edge cases

Check 1 — Check every release statement against proof

Procedure: Check every release statement against proof. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 2 — Prepare a reviewer demo with expected and failure journeys

Procedure: Prepare a reviewer demo with expected and failure journeys. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

Check 3 — Publish open-decision log and sign-off slots without fabricating signatures

Procedure: Publish open-decision log and sign-off slots without fabricating signatures. Compare the actual artifact or runtime output against the selected controlled baseline, capture the exact observed value and record any difference. Do not claim PASS if no execution evidence exists.

Expected acceptance evidence: a reproducible artifact reference (file, page, endpoint capture, SQL result, screenshot or test log), version/date, reviewer, and explicit PASS / FAIL / BLOCKED / NOT RUN classification.

C. Required outputs and closure evidence

- Readiness certificate marked conditional — assigned to Team lead / Member 6. Status: proposed/pending verification unless supplied as executable evidence.
- Demonstration checklist — assigned to Team lead / Member 6. Status: proposed/pending verification unless supplied as executable evidence.
- Open decision approval register — assigned to Team lead / Member 6. Status: proposed/pending verification unless supplied as executable evidence.
- Academic submission manifest — assigned to Team lead / Member 6. Status: proposed/pending verification unless supplied as executable evidence.

D. Implementation-facing acceptance rule

Do not close this chapter solely because its proposed design is documented. Closure requires agreement between affected members, updated source artifacts with version references, and evidence for all relevant contract and negative test cases. Unapproved decisions remain OPEN.

PART III — API ENDPOINT VERIFICATION CARDS

E-01 — GET /api/v1/health

Purpose: Health/readiness. Source authority: Member 4 API Specification v1.1, sections 4–5. Route shown here is a checklist transcription pending final confirmation where alternative path naming exists.

Request contract: Check method, path parameters, content type, required/optional fields, nullability and field names.

Successful response: Validate status code, payload schema, normalized enums and timestamp representation.

Invalid response: Cover invalid ID, missing record, invalid content, conflict and dependency failure when relevant.

Persistence: Confirm read/write mapping and transaction behavior; for GET ensure no unintended mutation.

Specific integration risk: Check response indicates system readiness rather than only process liveness.

Execution status: PLANNED / NOT RUN. No live endpoint test result was supplied.

E-02 — GET /api/v1/categories

Purpose: Supported categories. Source authority: Member 4 API Specification v1.1, sections 4–5. Route shown here is a checklist transcription pending final confirmation where alternative path naming exists.

Request contract: Check method, path parameters, content type, required/optional fields, nullability and field names.

Successful response: Validate status code, payload schema, normalized enums and timestamp representation.

Invalid response: Cover invalid ID, missing record, invalid content, conflict and dependency failure when relevant.

Persistence: Confirm read/write mapping and transaction behavior; for GET ensure no unintended mutation.

Specific integration risk: Verify empty and frozen five-class datasets and category id/code consistency.

Execution status: PLANNED / NOT RUN. No live endpoint test result was supplied.

E-03 — POST /api/v1/listings/analyze

Purpose: Submit and analyze. Source authority: Member 4 API Specification v1.1, sections 4–5. Route shown here is a checklist transcription pending final confirmation where alternative path naming exists.

Request contract: Check method, path parameters, content type, required/optional fields, nullability and field names.

Successful response: Validate status code, payload schema, normalized enums and timestamp representation.

Invalid response: Cover invalid ID, missing record, invalid content, conflict and dependency failure when relevant.

Persistence: Confirm read/write mapping and transaction behavior; for GET ensure no unintended mutation.

Specific integration risk: Multipart intake, input validation, new listing/run creation and typed results.

Execution status: PLANNED / NOT RUN. No live endpoint test result was supplied.

E-04 — GET /api/v1/listings/{listing_id}

Purpose: Get listing. Source authority: Member 4 API Specification v1.1, sections 4–5. Route shown here is a checklist transcription pending final confirmation where alternative path naming exists.

Request contract: Check method, path parameters, content type, required/optional fields, nullability and field names.

Successful response: Validate status code, payload schema, normalized enums and timestamp representation.

Invalid response: Cover invalid ID, missing record, invalid content, conflict and dependency failure when relevant.

Persistence: Confirm read/write mapping and transaction behavior; for GET ensure no unintended mutation.

Specific integration risk: Return immutable input snapshot and ensure no filesystem path leaks.

Execution status: PLANNED / NOT RUN. No live endpoint test result was supplied.

E-05 — POST /api/v1/listings/{listing_id}/analyses

Purpose: Re-analyze. Source authority: Member 4 API Specification v1.1, sections 4–5. Route shown here is a checklist transcription pending final confirmation where alternative path naming exists.

Request contract: Check method, path parameters, content type, required/optional fields, nullability and field names.

Successful response: Validate status code, payload schema, normalized enums and timestamp representation.

Invalid response: Cover invalid ID, missing record, invalid content, conflict and dependency failure when relevant.

Persistence: Confirm read/write mapping and transaction behavior; for GET ensure no unintended mutation.

Specific integration risk: Create new run without overwriting previous evidence.

Execution status: PLANNED / NOT RUN. No live endpoint test result was supplied.

E-06 — GET /api/v1/listings/{listing_id}/analyses

Purpose: List runs. Source authority: Member 4 API Specification v1.1, sections 4–5. Route shown here is a checklist transcription pending final confirmation where alternative path naming exists.

Request contract: Check method, path parameters, content type, required/optional fields, nullability and field names.

Successful response: Validate status code, payload schema, normalized enums and timestamp representation.

Invalid response: Cover invalid ID, missing record, invalid content, conflict and dependency failure when relevant.

Persistence: Confirm read/write mapping and transaction behavior; for GET ensure no unintended mutation.

Specific integration risk: Test ordered run history and failure state visibility.

Execution status: PLANNED / NOT RUN. No live endpoint test result was supplied.

E-07 — GET /api/v1/analyses/{analysis_run_id}

Purpose: Get analysis. Source authority: Member 4 API Specification v1.1, sections 4–5. Route shown here is a checklist transcription pending final confirmation where alternative path naming exists.

Request contract: Check method, path parameters, content type, required/optional fields, nullability and field names.

Successful response: Validate status code, payload schema, normalized enums and timestamp representation.

Invalid response: Cover invalid ID, missing record, invalid content, conflict and dependency failure when relevant.

Persistence: Confirm read/write mapping and transaction behavior; for GET ensure no unintended mutation.

Specific integration risk: Check nested evidence, nullable fields, finding and suggestion schemas.

Execution status: PLANNED / NOT RUN. No live endpoint test result was supplied.

E-08 — POST /api/v1/suggestions/{suggestion_id}/decision

Purpose: Seller decision. Source authority: Member 4 API Specification v1.1, sections 4–5. Route shown here is a checklist transcription pending final confirmation where alternative path naming exists.

Request contract: Check method, path parameters, content type, required/optional fields, nullability and field names.

Successful response: Validate status code, payload schema, normalized enums and timestamp representation.

Invalid response: Cover invalid ID, missing record, invalid content, conflict and dependency failure when relevant.

Persistence: Confirm read/write mapping and transaction behavior; for GET ensure no unintended mutation.

Specific integration risk: Accept/Edit/Ignore, only one persisted decision and conflict behavior.

Execution status: PLANNED / NOT RUN. No live endpoint test result was supplied.

E-09 — GET /api/v1/review-queue

Purpose: Review queue. Source authority: Member 4 API Specification v1.1, sections 4–5. Route shown here is a checklist transcription pending final confirmation where alternative path naming exists.

Request contract: Check method, path parameters, content type, required/optional fields, nullability and field names.

Successful response: Validate status code, payload schema, normalized enums and timestamp representation.

Invalid response: Cover invalid ID, missing record, invalid content, conflict and dependency failure when relevant.

Persistence: Confirm read/write mapping and transaction behavior; for GET ensure no unintended mutation.

Specific integration risk: Verify latest usable run selection, filtering and queue-state consistency.

Execution status: PLANNED / NOT RUN. No live endpoint test result was supplied.

E-10 — GET /api/v1/listings/{listing_id}/image

Purpose: Image retrieval. Source authority: Member 4 API Specification v1.1, sections 4–5. Route shown here is a checklist transcription pending final confirmation where alternative path naming exists.

Request contract: Check method, path parameters, content type, required/optional fields, nullability and field names.

Successful response: Validate status code, payload schema, normalized enums and timestamp representation.

Invalid response: Cover invalid ID, missing record, invalid content, conflict and dependency failure when relevant.

Persistence: Confirm read/write mapping and transaction behavior; for GET ensure no unintended mutation.

Specific integration risk: Safe image access and expected content type.

Execution status: PLANNED / NOT RUN. No live endpoint test result was supplied.

PART IV — FR-001 TO FR-025 TRACEABILITY NOTES

Transcribed source is a Member 2 secondary matrix, not the independently supplied original FRS; no direct stakeholder approval is inferred.

FR entry 01

FR-001 | Product Image Upload | The system shall allow a seller to provide one fashion product image for listing analysis. | Must Have | product_listing | image_ref, image_origin | Snapshot image reference; actual file-validation process is transient. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 02

FR-002 | Product Title and Description Input | The system shall accept a product title and a product description as text for attribute-claim extraction. | Must Have | product_listing | title, description, text_origin | Original text remains available for review. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 03

FR-003 | Product Category Input | The system shall accept the seller-provided product category and make it available to the unsupported-category check. | Must Have | product_listing | seller_category | Raw category retained even if unsupported. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 04

FR-004 | Product Type Classification | The system shall predict product type within the five EDA-selected categories using the ResNet/MobileNet main CV model, with CLIP zero-shot maintained as baseline and fallback. | Must Have | visual_prediction; supported_category | attribute, predicted_category_code, model_source, confidence | Primary and CLIP evidence can be compared; weights remain external. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 05

FR-005 | Color Recognition | The system shall predict product color from the image, with a dominant-color check used to cross-check the visual color decision. | Must Have | visual_prediction | attribute, predicted_color, confidence, model_source | Color evidence and optional cross-check. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 06

FR-006 | Image Quality Analysis | The system shall perform basic checks of image resolution, blur and brightness and make relevant quality findings available to review and, when the optional Listing Quality Score is enabled, to scoring. | Must Have | analysis_run; analysis_finding | image_width_px, image_height_px, is_blurred, brightness_status, reason_code | Summaries persisted; raw pixel transforms transient. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 07

FR-007 | NLP Claim Extraction | The system shall normalize title and description text and extract only product-type and color claims using a controlled vocabulary, rules and fuzzy matching. | Must Have | text_claim | attribute, source_field, matched_text, canonical_value | Structured claims; NLP normalization computations transient. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 08

FR-008 | English/Arabic Script Detection | The system shall identify Arabic characters to select the small fixed Arabic lexicon and otherwise apply the English vocabulary. | Must Have | text_claim | script_route | Records lexicon routing for extracted claims; per-token detection transient. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 09

FR-009 | Image-Text Consistency Checking | The system shall compare image-supported product type and color with claims extracted from title and description using rules and a lightweight mismatch classifier. | Must Have | analysis_finding; visual_prediction; text_claim | reason_code, visual_value, claimed_value | Stores material differences; comparisons computed by application. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 10

FR-010 | Product Type Mismatch Detection | The system shall flag contradictions between visual product type and text claims and distinguish same-group from different-group contradictions. | Must Have | analysis_finding | TYPE_MISMATCH, severity, explanation | Type contradiction evidence. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 11

FR-011 | Color Mismatch Detection | The system shall apply color-family and near-shade rules to flag different-family color contradictions while allowing minor or ignored near-shade differences. | Must Have | analysis_finding | COLOR_MISMATCH, severity, explanation | Color family contradiction evidence. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 12

FR-012 | Missing Attribute Detection | The system shall identify when an image-supported product type or color is absent from the submitted text rather than contradicted by it. | Should Have | analysis_finding | MISSING_ATTRIBUTE, attribute_scope | Optional finding if supporting feature retained. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 13

FR-013 | Confidence Scores | The system shall expose confidence for visual predictions and mismatch findings, distinguishing confidence from verified correctness. | Must Have | visual_prediction; analysis_finding | confidence | Stores available calibrated confidence, not truth guarantee. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 14

FR-014 | Low-Confidence Handling | The system shall flag predictions below the configured validation-derived confidence threshold and route affected listings to Needs Review rather than asserting certainty. | Must Have | analysis_finding; analysis_run | LOW_CONFIDENCE, queue_state | Threshold configured externally; decisions persisted. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 15

FR-015 | Unsupported Category Handling | The system shall flag suspected out-of-scope images when CV and CLIP disagree, the predicted type contradicts the seller category, or CLIP favors the other-product prompt, avoiding a forced definitive type. | Must Have | analysis_finding; analysis_run | UNSUPPORTED_CATEGORY, queue_state | Supported category remains controlled and unknown until EDA. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 16

FR-016 | Reason Codes | The system shall associate applicable findings with the proposal-defined reason codes and a plain-language explanation template. | Must Have | analysis_finding | reason_code, explanation | Six allowed codes and nonblank explanation. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 17

FR-017 | Mismatch Severity Classification | The system shall assign severity using the approved product-group and color-family rule table. | Must Have | analysis_finding | severity | Rule application is transient; resulting severity persisted. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 18

FR-018 | Listing Quality Score | The system shall calculate a transparent 0–100 Listing Quality Score from image quality, attribute completeness and image-text consistency only. | Should Have | analysis_run | quality_score | Optional 0–100 output; SQL allows NULL if deferred. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 19

FR-019 | Listing Inconsistency Level | The system shall calculate a heuristic Listing Inconsistency Level from the number and severity of Major and Critical mismatches. | Must Have | analysis_run | inconsistency_level | Low/Medium/High result recorded; rule computation external. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 20

FR-020 | Review Queue Assignment | The system shall assign each analyzed listing to the proposal-defined review status based on inconsistency, low confidence, image blur, missing attributes and unsupported-category flags. | Must Have | analysis_run | queue_state | Queue derived from latest run; no redundant queue table. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 21

FR-021 | Template-Based Suggested Fix | The system shall propose a corrected title and description through controlled templates grounded only in supported type and color predictions, without using an LLM. | Must Have | suggested_correction | suggested_title, suggested_description | Template output stored; template files remain external. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 22

FR-022 | Before/After Comparison | The system shall present original and proposed listing text side by side and indicate what changed and why. | Must Have | product_listing; suggested_correction | original title/description; suggested_title/description | UI comparison generated from saved snapshots. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 23

FR-023 | Accept Suggestion | The system shall allow the seller to accept a presented correction explicitly without automatically publishing the listing. | Must Have | seller_decision | decision=accept, decided_at | Explicit acceptance with no publish capability. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 24

FR-024 | Edit Suggestion | The system shall allow the seller to modify the proposed text before approving their chosen wording, without autonomous publication. | Must Have | seller_decision | decision=edit, final_title, final_description | Stores seller-approved edited wording without changing original. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.

FR entry 25

FR-025 | Ignore Suggestion | The system shall allow the seller to ignore a suggested correction and retain the original listing text for that decision. | Must Have | seller_decision; product_listing | decision=ignore; original title/description | Unapplied suggestion; original input retained. | Proposed — needs integration validation

Verification: independently compare the requirement against member 1 UML, member 3 component mapping, member 2 persistence, member 4 endpoint/schema, member 5 UI state and the Member 6 planned test matrix. Status remains NOT VERIFIED until source baseline and execution evidence are obtained.